

Запускать так: `./run.sh`

По необходимости доставить библиотеку с решателем ЛП (в решении есть только солвер непрерывной ЛП, не целочисленной)

```
sudo apt install coinor-libclp-dev
```

Для просмотра второго и третьего решения в полях BackpackSolver в `main.cpp` надо поменять `sMaxMemory` с $1 \ll 27$ на $1 \ll 20$, а также законментировать первое решение.

Ибарра-Ким с дп

Сперва было написано стандартное решение рюкзака на $O(nW)$ с алгосов 2 семестра. То есть заводилась таблица размера $n \times W$ и в $dp[i][j]$ указывалась максимальная стоимость, которую можно уместить в рюкзак размера W используя первые n объектов. Для такого алгоритма выполнена корректность в том смысле, что ответ действительно даёт максимальную стоимость. В реализации выделяется nW памяти под инты, после чего делается 2 пробежки по выделенному массиву, будем считать, что затрачивается $4nW$ бит и nW времени. Чтобы укладываться в лимиты, будем считать, что не хорошо выходить за 0.5 гб памяти, что значит $4nW \leq 2^{29} \Leftrightarrow nW \leq 2^{27}$, так как количество операций мы считаем nW , то этого же ограничения и хватит для работы за не слишком большое время, поэтому им и ограничимся. В самой первой реализации было написано за $O(nW)$, если $nW \leq 2^{29}$ и ничего не класть в рюкзак иначе. Такое решение прошло большие пороги на 1, 2, 3, 5 тестах, поскольку решение за nW находит точный ответ¹, то пойдя по такому пути мы дополнительно выводим что ответ точный, итого 24 балла 6,6,6,0,6,0.

Далее была написана смесь алгоритма на подобиИ Ибарра-Ким и решения через dp. Возьмём $c = \lceil \frac{nW}{2^{27}} \rceil$ и увеличим вес каждого груза так, чтобы он делился на c . Поделим всё на НОД, продумаем алгоритм за nW , найдём сколько места осталось в рюкзаке, всё оставшееся место дополним алгоритмом за nW' , W' — меньше чем W , так что так может получиться. Иначе предварительно повторим процедуру, выбрав для остатка c побольше, чтобы ускорять последующие шаги и уложиться во время. Этот подход дал пройти четвёртый тест на 5, итого 29: 6,6,6,5,6,0.

В этом решении мы сперва отказались от увеличения c бездумно: скорее всего мы что-то протолкнули в массив в первый раз, так что алгоритм будет работать быстрее и так. То есть сперва мы пытаемся не менять c . Если с одним c не вышло, то попробуем сперва увеличить c на 1, что изменит остатки и даст элементам протолкнуться по другому. Также заметим, что на первой итерации мы находим довольно грубое приближение. Так что разумно было бы брать не все такие объекты, а часть из них, а если остаток был оптимальный то мы найдём его после ещё одной итерации. Такая константа должна быть не слишком велика, чтобы не забить рюкзак неоптимальными решениями и при этом не слишком мала, поскольку рюкзак, хоть и округлённый, однако для конкретного округления видит как стыковать блоки, а при меньшем это решение потеряется. Также слишком маленькая константа может застопорить алгоритм.

Выбрав $\alpha = 0.25$ были пройдены все 6 тестов на максимум. Метрики составили

99798, 142156, 100236, 3967168, 109899, 1099883.

Что дало 34 балла так: 6, 6, 6, 5, 6, 5. Отныне будем пытаться оптимизировать память. На деле наше решение не столь плохо с ней и сейчас. Если заменить $nW \leq 2^{27}$ на $nW \leq 2^{20}$, то

¹Ну это мы наверное знаем, потому что сдали на прошедшем курсе алгосов

метрики составят

99798, 142156, 100236, 3967173, 109899, 1099128.

То есть на 1, 2, 3, 5 тестах будут получены точные ответы, на остальных ответы немного ухудшатся. При этом на таких значений алгоритм ни на одном тесте не пользуется динамикой за nW в чистом виде, получая метрики 5, 5, 5, 5, 5, 0, в сумме 25, хоть мы и знаем, что 4 раза он нашёл настоящий оптимум.

Далее попробуем другой подход.

Линейные релаксации

На семинаре была разобрана идея как пытаться получить идеальную постановку ЦЛП. Сперва выпишем саму постановку: мы хотим найти

$$\max_{x \in \{0,1\}^n} \sum_{i=0..n-1} v_i x_i, \text{ s.t. } \sum_{i=0..n-1} x_i w_i \leq W.$$

Сперва решим линейную релаксацию, если она даст невалидное решение, где $x_i \notin \mathbb{Z}$, пусть это решение имеет $x_{i_1}, \dots, x_{i_m} > 0$, среди них первые несколько самых выгодных взяты целиком, оставшийся дробно, это невалидно и установим $\sum_{j=1}^m x_{i_j} \leq m-1$. Попробуем перерешать задачу, и каждый раз находя невалидное решение будем выбрасывать из него по минимум, чтобы получить валидное и запоминать его как возможное, в конце выведем лучшее.

Мы временно отключили динамику совсем чтобы оценить наше решение. Сперва это не заработало, такого условия может не хватать: мы получали несколько объектов, один из которых взят почти целиком, оставшиеся — маленькими кусками, а именно из нескольких объектов одной и той же удельной стоимости они все взяты на 0.05, то есть ограничение на $\leq m-1$ они и так проходили.

Если добавив обычное условие мы не получили результата, то мы будем добавлять более сложные условия:

- Возьмём решение, которое мы нашли этой релаксацией. Если к выбранным объектам нельзя добавить ни одного другого, то они в любом случае отыграли своё, можно запретить брать их всех сразу. Это прошло первый тест на 5.
- Возьмём все объекты с 1, если можем, то дорешаем через дп, обновим решение, а после, запретим брать этот набор всеми единицами. Благодаря этому симплекс сошёлся на первом тесте, в смысле, что нашими отсечениями без потери целочисленных вершин он нашёл целочисленную точку.

При такой реализации симплекс уже стал получать хорошие линейные релаксации. А именно метрики составили

99798, 142156, 100236, 3967120, 109899, 1099881.

Где симплекс был уверен в первом решении, то есть было набрано 31: 6, 5, 5, 5, 5, 5.

Проблема этого решения заключается в том, что просто в какой-то момент он дорешал пустое количество единиц, и при уменьшении памяти не находит таких решений. Поэтому продолжим искать хорошие эвристики.

Нынешнее решение на втором тесте достаточно быстро проходит в релаксацию с ненулевыми значениями 0.44, 0.44, 0.44, 0.44, 0.55, 1, 1. Здесь появились объекты с суммой весов равной 1. Это значит что из пары можно взять не более чем один такой объект. На примере

есть один более тяжёлый. Попытаемся взять его, дорешать, а после — запретить одновременно брать его и единицы.

Это не помогло, алгоритм нашёл более сложный способ толкать такие решения, уменьшив число единиц и сделав более хаотичные дробные числа. На этом этапе при $nW \leq 2^{20}$ метрики составили

99798, 141724, 68640, 3838316, 101334, 1097042.

Что даёт 5 баллов: 5, 0, 0, 0, 0, 0.

Потеряем корректность, чтобы выйти из этой проблемы. Если некоторое число итераций числа с ненулевыми коэффициентами остались ненулевыми, то напишем их сумму значений и вычтем не 1, а 2, после 3 и так далее, пока не дойдём до нуля. Реализуем это так: будем записывать количество раз, которое мы встречали конкретный набор из ненулей. Если оно было хотя бы kC , то вместо $\sum_{j=1}^m x_{i_j} \leq m - 1$ напишем $\sum_{j=1}^m x_{i_j} \leq m - k$.

Мы перестали давать одинаковые решения и подняли значение метрик на 4,5,6 тесте. Метрики составили

99798, 141724, 68640, 3926960, 109894, 1099157.

Это 8 баллов 5, 0, 0, 0, 3, 0 при той же памяти в $nW \leq 2^{20}$.

Локальный поиск

Линейной релаксацией мы получили стартовое решение, попытаемся улучшить его методами локального поиска.

Удалим один предмет из имеющегося рюкзака и, если нам хватает памяти, то заполним это место с помощью дп, иначе — добавим несколько объектов жадно, пока памяти не начнём хватать, после чего добавим с помощью дп.

Такое улучшение было принято на каждом тесте не более одного раза, однако улучшили метрики. Они составили

99798, 141724, 100062, 3966753, 109894, 1099873.

То есть 14 баллов 5, 0, 3, 0, 3, 3 при $nW \leq 2^{20}$.

Сделаем больше возможности для ходьбы: напишем табу поиск, а именно разрешим про-талкивать элемент, даже если он делает ухудшение, но суммарное ухудшение за все итерации не должно превышать некоторого процента от лучшего известного ответа. Если элемент удаляли недавно, то повторно удалять его не будем, за исключением случаев, когда это найдёт ответ, лучше, чем за всё время.

Возникли следующие параметры: *tabu_bad* — насколько можно быть хуже, чем наилучшее найденное решение; *tabu_max_iter* — на протяжении какого числа шагов нельзя прикасаться к элементу и параметры того, на сколько итераций запускать каждую процедуру.

Сперва было попробованно ставить фиксированный *tabu_bad*, но при значениях близких к 1 (порядка 0.8 – 0.9) не удавалось выйти из локального минимума, и решение давало 14-16 баллов, на слишком маленьких *tabu_bad* (порядка 0.6) проблема была обратная — решение не пыталось идти к локальному минимуму, на ответах с большим количеством маловесомых объектов табу поиск одобрял практически любые изменения.

Таким образом маленький *tabu_bad* хорошо подходил для локального минимума, выбраться из которого можно только сильно уменьшив функцию, а большое *tabu_bad* чуть расширяет обычный локальный поиск спокойно обходя не глубокие локальные минимумы.

Ещё одна проблема маленького `tabu_bad` была в том, что если запускаться на нём и не менять решение, то для больших `tabu_bad` алгоритм начнёт из точки далёкой от оптимума, и не будет делать полезные улучшения, запуская жадник на заведомо плохом решении.

Было принято решение запускаться на многих `tabu_bad` по очереди, а также перед перезапуском отбрасывать решение до лучшего, что есть на данный момент.

Второй параметр `tabu_max_iter` влиял на программу следующим образом: при маленьком `tabu_max_iter` если удаление какого-то легковесного предмета не сильно изменяет метрику, то алгоритм будет то и дело добавлять и убирать его делая неосмысленные изменения. При слишком большом же `tabu_max_iter` есть риск полной остановки программы: все объекты ещё слишком рано удалять.

Будем считать, что найденное к этому моменту линейной релаксацией и локальным поиском без `табу` решение в целом показывает то число предметов, которые надо удалять, и попробуем сделать `tabu_max_iter` пропорциональным числу предметов в лучшем решении. Подставив разные константы лучше всего метрики выходили при дважды числе объектов в найденном оптимальном рюкзаке.

К этому моменту метрики составили

99798, 141724, 100236, 3966862, 109894, 1099873.

Что даёт 19 баллов: 5, 0, 5, 3, 3, 3.

Основная проблема этого метода в том, что мы можем обработать только маленькие объекты, которые можно добавить, удалив какие-то побольше. Большие же объекты сами в рюкзак попасть не могут. Дадим локальному поиску другой тип изменений: насильно добавить предмет и убрать каких-то предметов, которые переполняют рюкзак.

В `табу` поиск внедрим эту идею естественным образом: сделаем параметр `add_alpha` — вероятность того запускаем мы шаг с удалением элемента или с добавлением. Низкие `add_alpha` хорошо подходят, если оптимальный ответ состоит из множества маленьких предметов, а большие если из нескольких тяжёлых. Поскольку структура ответа нам не известна — запустимся с разными и возьмём лучший из ответов.

Это существенно улучшило метрики на втором и пятом тесте и стало стабильнее работать на четвёртом.

Алгоритм получился с вероятностной корректностью и метрики различаются при перезапуске, медианные метрики составили:

99798, 142156, 100236, 3967116, 109899, 1099873

Что дало 28 баллов 5, 5, 5, 5, 5, 3 при запуске `дп` при $nW \leq 2^{20}$.

Итог

Решение с Ибарра-Ким с `дп` оказалось наилучшим, как минимум благодаря гарантии неухудшаемости ответа. Оно даёт 34 балла: 6, 6, 6, 5, 6, 5.

Также есть альтернативное решение, при меньшем числе памяти. Оно берёт все высокие тесты кроме одного, уменьшив память в 2^7 раз.